

OPERATING SYSTEMS PROJECT

# LINUX SERVICE & PROCESS SUPERVISOR

A practical supervisor for starting, monitoring and controlling concurrent Linux services.


Presented by

| | |
|---|---|
| K Saicharan | 2520030002 |
| K Prudhvi Sai | 2520030286 |
| P Laxmikanth Reddy | 2520030018 |

# Introduction



- Linux allows many processes and services to run at the same time.
- Managing all these processes manually can be difficult.
- A process may stop or crash while running.
- Our project helps to start, monitor, control, and restart services automatically.
- It also allows the user to check the current status of each service.




| AUTO START | MONITOR | DETECT | RESTART | CONTROL | STATUS |
|---|---|---|---|---|---|

# Abstract

The Linux Service and Process Supervisor is a C-based project used to manage multiple services running on a Linux system. It helps in starting services and continuously monitoring their processes while they are running. If a service stops or crashes, the supervisor detects it and can restart the service according to the restart limit given in the configuration file.

The project also provides a command-line tool called svctl, through which the user can start, stop, restart, pause, resume, and check the status of services. Communication between svctl and the main supervisor is handled using named FIFO. The project uses important Linux concepts such as fork(), execv(), waitpid(), signals, process control, and inter-process communication. It provides a practical way to understand how processes and services can be managed in Linux.



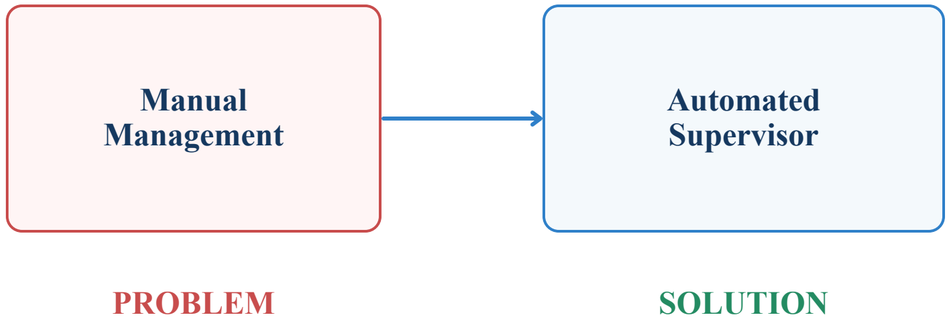
# Problem Statement



In Linux, many services and processes can run at the same time. Managing them manually makes it difficult to keep track of their current state and to handle processes when they stop or crash.

The project aims to develop a Linux Service and Process Supervisor that can:

- Start and monitor multiple services.
- Detect when a service stops or crashes.
- Restart a failed service up to a configured limit.
- Allow the user to start, stop, restart, pause, and resume services.
- Display the current status of each service.
- Use Linux process-management and inter-process communication concepts to manage the services efficiently.




**Detect • Restart • Control • Report**

# Objectives



01 Start and monitor multiple services

02 Detect terminated or failed processes

03 Auto-restart failed services

04 START • STOP • RESTART • PAUSE • RESUME

05 Track status and restart counts

06 Demonstrate process management and IPC

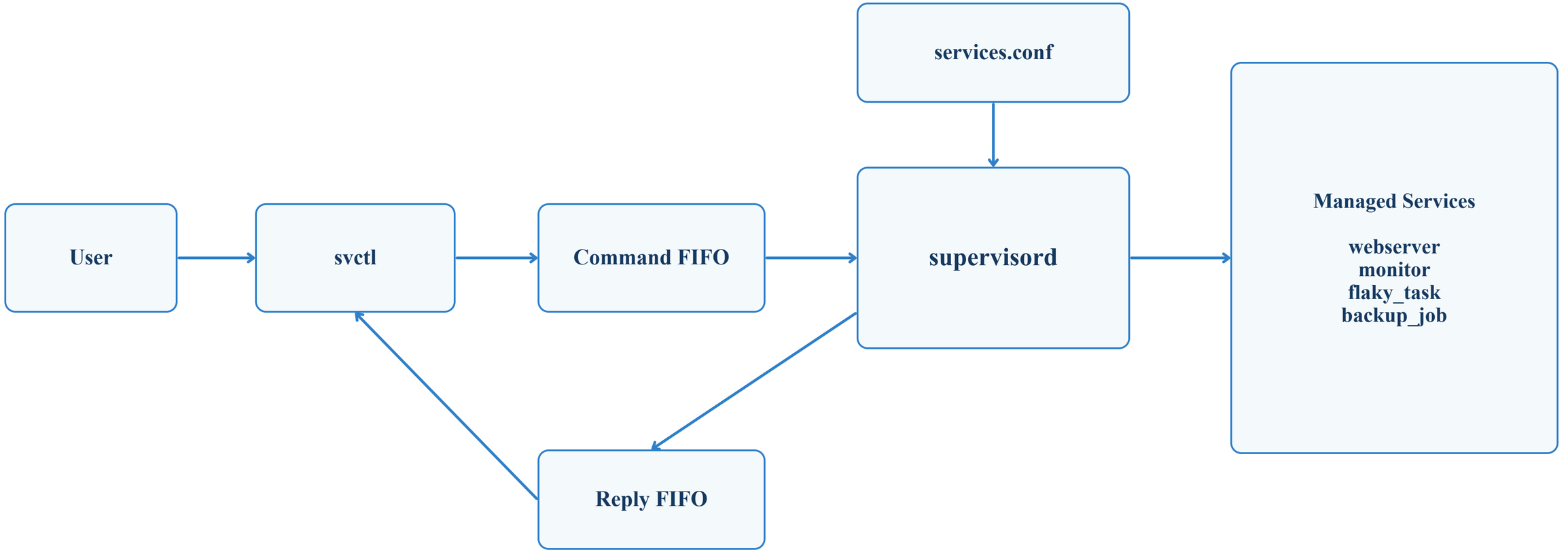
# Concepts Used



| API / CONCEPT | PURPOSE |
| --- | --- |
| `fork()` | Creates a child process |
| `execv()` | Executes the service program |
| `waitpid()` | Monitors child process status |
| `SIGCHLD` | Notifies on child termination |
| `kill()` | Sends signals to processes |
| `FIFO` | IPC mechanism between processes |

# System Architecture






Commands flow right; status replies return through a dedicated FIFO.

# Technologies Used



### C
Programming language

### VS Code
Source editing

### Ubuntu Linux
Development platform

### POSIX APIs
Process & signal control

### GCC
Compiler toolchain

### Named FIFO
Inter-process communication

### GNU Make
Build automation

# Advantages



- ✓ **Automatic process monitoring**
- ✓ **Automatic restart of failed services**
- ✓ **Easy runtime control via svctl**
- ✓ **Configurable restart limits**
- ✓ **Simple FIFO-based IPC**
- ✓ **Practical demonstration of Linux OS concepts**

# Conclusion



The Linux Service and Process Supervisor provides a simple way to manage multiple services in a Linux system. It can start and monitor processes, detect when a service stops or crashes, and restart it based on the configured limit. The svctl program also allows users to control and check the status of services.

Overall, the project gives practical understanding of Linux process management, signals, child-process monitoring, and FIFO-based inter-process communication.



# THANK YOU

LINUX SERVICE & PROCESS SUPERVISOR • OPERATING SYSTEMS PROJECT